



Transaction Concepts (follow the book)

What is Transaction?

- you may hear a term very often at work and people refer to many meaning, nowadays we have standard meaning

S	S #	SNAME	STATUS	CITY	SP	S #	P #	QTY
	S 1	SMITH	20	LONDON		S 1	P 1	300
	S 2	JONES	10	PARIS		S 1	P 2	200
	S 3	BLAKE	30	PARIS		S 1	P 3	400
	S 4	CLARK	20	LONDON		S 1	P 4	200
	S 5	ADAMS	30	ATHENS		S 1	P 5	100
P	P #	PNAME	COLOR	WEIGHT	CITY			
	P 1	Nut	Red	12	London			
	P 2	Bolt	Green	17	Paris			
	P 3	Screw	Blue	17	Rome			
	P 4	Screw	Red	14	London			
	P 5	Cam	Blue	12	Paris			
	P 6	Cog	Red	19	London			

The supplier-and-parts database

- S table represent object instances or entity instances of supplier.(master table)
- P represent object instances or entity instances of part. (master table)
- SP represent relationship instance between supplier and part. (transaction table) —

The term "transaction" has several meanings, depend on the popularity at the time.

-  A transaction could mean a table which keeps relationship instances (such as the SP table).
 - Each row is called 1 transactions (surprisingly). → not actually correct (1970, 1980)
-  A transaction could refer to a file which keeps changes (master file, transaction file).
 - Historical part of transaction
 -  (1960s) → Master file, and because the tape must be processed sequentially → sorted by a key → so you can search for a key, or else you have to search the entire tape.
 - How to modified this one? (physically at that time it implement on tape.) (sequential sorted by the key)
 - How you insert the data? (insert new records?)
 - How to delete old record?
 - How to modify existing record?
 - Answer: We can't just directly modify the data, we can't do that, we have to keep the changes somewhere → Transaction file(Tx)
 - Tx file → A file which keeps changes to be made to the master file.
 - Example, you have a task master file, inventory master file sorted and you would like to insert, update, delete the inventory → you have to collected those changes into this file (transaction file). **The record must be sorted by the same key as master file.**

The idea is you collect all changes and sort by the same key as master file, and periodically which maybe the end of the day, every hour, depends → you open Master file(old master) and transaction file

Sequential file processing → you open both file, then compare a key, if master file key is less than in value to transaction file key, it means you don't have to do anything → you just write new record to the new master file. (old master, Tx, new master)

- if the key match, it could be a modify or a delete.
 - if you want to delete some row → you record the key, the value of the key of the row to be deleted into Tx, and when it matches, you simply not write the old record to the new one.
 - If it modify → you write a new value to the new master file
 - insert → the key must be different from the one in the old master

By checking record, if the key match. key don't match could be insert or error

- (this very idea is used on blockchain as well, blockchain is sequential)



as conclusion, if you ask people in the 60s, what a transaction is? A transaction is kind of file which keep changes to be made to the master file. (this is transaction in the past)

- 📌 1970s-1980s → we have disks, and disk is not sequential like tape → it's direct access.
 - the term transaction is slightly different from the previous one because it direct access, you don't have to accumulate changes into Tx → you can go direct to the place and update the particular record you want to update.
 - 🌱 A transaction is a single database operation.
- What about money transfer? (withdraw from one, deposit to another one) how many transaction for money transfer activity?

- 1 - withdraw and deposit should go together
- 2?

nowadays, some people also called single database operation as transaction → not quite correct.

- 📌 1990s → present (definition we used through out lecture in this semester.

- 🌱🌱 A transaction is a logical unit of work in which integrity constraints are allowed to be violated. (current day, modern day transaction)
- What is mean by logical unit of work? (LUW)
 - A logical unit of work is a group (has order) of database operations. (you may have insert, delete, modify operation)
 - for example, transfer of money → you want to move an amount from account A to account B. (transfer from A to B x bath)

$$\text{Acct } A \xrightarrow{x} \text{Acct } B$$

- in that money transfer activity, in fact, you have withdrawal x bath from A.
- deposit x bath to b
- grouping them → Transaction → Logical unit of work.

$$\text{Logical unit of work} \left[\begin{array}{l} A := A - x \\ B := B + x \end{array} \right.$$

- Example using SQL

Acct

Acct#	...	Amount
-------	-----	--------

Update Acct
 Set amount = amount - x
 where Acct# = 'A'

Update Acct
 Set amount = amount + x
 where Acct# = 'B'

- to mark the begin and the end of transaction
 - Sync points - sync points mark the begin and end of transactions

begin Tx
 A := A - x
 B := B + x
 end Tx

Acct

Acct#	...	Amount
-------	-----	--------

Sync points

begin Trans

Update Acct
 Set amount = amount - x
 where Acct# = 'A'

Update Acct
 Set amount = amount + x
 where Acct# = 'B'

- end Trans

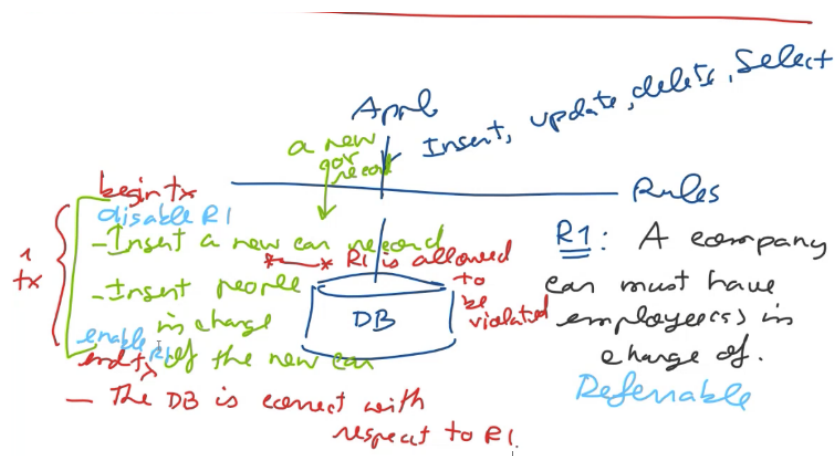
mark the begin & end of transaction

- the syntax on the sync point depends on language or tool you using.
- What is integrity constraints? why we need concept of integrity constraint here?
 - An integrity constraint or integrity rule or validation rules → are rule that enforce the correctness of the database

- You may have rule like student may borrow no more than 10 books, you can't enroll in 2 course that have the same exam time.
- Where should we keep this rule
 - (possible) part of application program, but if the rule change you have to go back to application program and change them. Most people don't want to touch source code unnecessary.
 - nowadays, most integrity constraint are declare on database. And those rule enforce by DBMS.
 - you may declare student can't borrow more than 5 books, and let the DBMS check upon the insertion of borrow record. (if exceed → reject the insertion)

They should be declare on the database and be enforced by the DBMS → their command like create constraint, create validation rule.

Example



insert, update, delete, select → the operation trigger the rule, if it follow it can be done.

by R1 → you can't have a car without employee take care of it

what if you buy a new car → new car can't be inserted to the system?

- what should we do to insert a new car?
 - Insert new car record
 - Insert people in charge of the new car
 - (there are 2 operation and we group them together → we allow R1 to be violated)
- without this concept of transaction - you can't insert a new car.
- the implementation of this concept
 - a very primitive way(manually) → begin with like disable R1, before the end like enable R1.
 - a good and expensive system → you declare rule, you code the rule → put keyword like deferrable. Deferrable mean you don't find the rule inside, after transaction finish, you enforce the rule. R1 is allow to be violated inside, but outside the database is correct with respect to R1.

-
- Most recent product like Microsoft or oracle → True
 - IBM avoid using the term transaction (old company) → They use LUW.
-

Sync point in SQL

- There are sync point in sql, design to mark the begin and the end of transaction

commit [work (option)]

Sync points in SQL

- 1) commit [work]
- 2) Rollback [work]
- 3) Set autocommit off

- Some people (don't know sync point) think commit, rollback is simple sql command → noooo

- commit: database operations from the previous sync point to the commit point are confirmed
 - for example, you doing many activities, and after you commit → the data from commit to previous sync point are confirmed.
- roll back: (opposite of commit) Database operations from the previous sync point to the rollback point are cancelled.
- you can use this commit and roll back command in an interactive mode as well
- set auto commit off (opposite to set auto commit on)
 - The opposite to set auto commit on which forces each operations to commit individually
 - auto commit off - each operation will not be committed. There needs to be an explicit commit operation to confirm the database operations.
 - (what is set auto commit on - after each single sql operation, the operation is committed automatically)

To do transaction processing, we have to set auto commit off

```

Set autocommit off
Update ACCT
Set amount = amount - 20
where Acc# = 'A'
Update ACCT
Set amount = amount + 20
where Acc# = 'B'
commit work

```

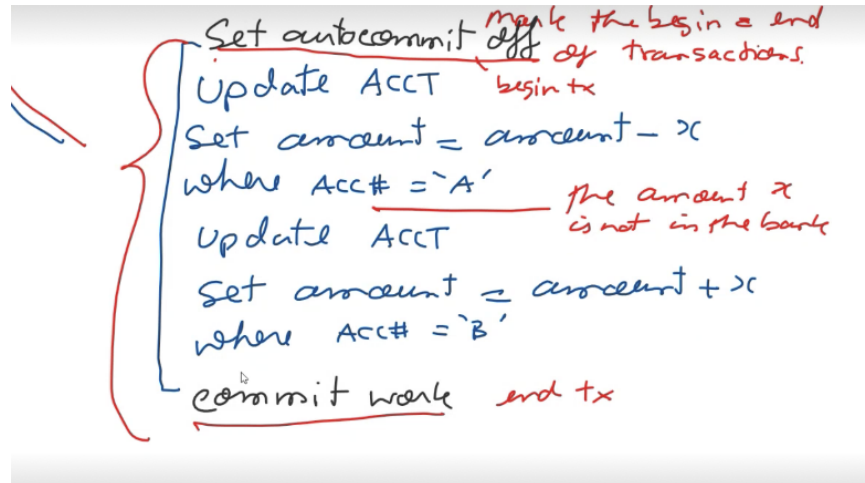
this is a transaction

Instead of using begin transaction and end transaction, we can use set autocommit off, commit

The rule can be explicit or implicitly declare.

- when transfer money

- implicit → no amount should be lost during transfer.



- amount x disappear for a short period and arrived back. This a little integrity constraint that allowed to be violated during transaction here.

Definition from a book

Transaction Concept

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- A transaction must see a consistent database.
- During transaction execution the database may be temporarily inconsistent.
- When the transaction completes successfully (is committed), the database must be consistent.
- After a transaction commits, the changes it has made to the database persist, even if there are system failures.
- Multiple transactions can execute in parallel.
- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions

read in here too.

A completion of transaction definition must include 4 bullets. (equivalent to our definition, if ask can refer to this)

- A transaction is a unit of program execution that accesses and possibly updates various data items - definition from the book, not yet complete

- A transaction must see a consistent database - it means that before the transaction, the database is assumed to be correct with respect to integrity constraint.
 - consistent - means correct with respect to integrity constraint.
- During transaction execution the database may be temporarily inconsistent.
 - during execution of transaction, some rule may be violated, and the database becomes temporarily inconsistent.
- When the transaction completes successfully (is committed), the database must be consistent.

What if there's a failure occurs, what does the system do to ensure transaction correctness? (2 main issues)

- How do you know the result is correct.

What actually is meant by commit



commit: Database operations from the previous sync point to the commit point are confirmed

** it not actually said it write to the disk yet, it just said it confirmed.

Commit is a logical statement, logical level confirmation statement from the DBMS

Some example of wrong

begin tx

$A := A - x$

commit ? x x x

$B := B + x$

end tx

money transfer will be successful if withdraw + deposit both successful

- if withdrawal commit and deposit failed, you can't roll back the whole thing.

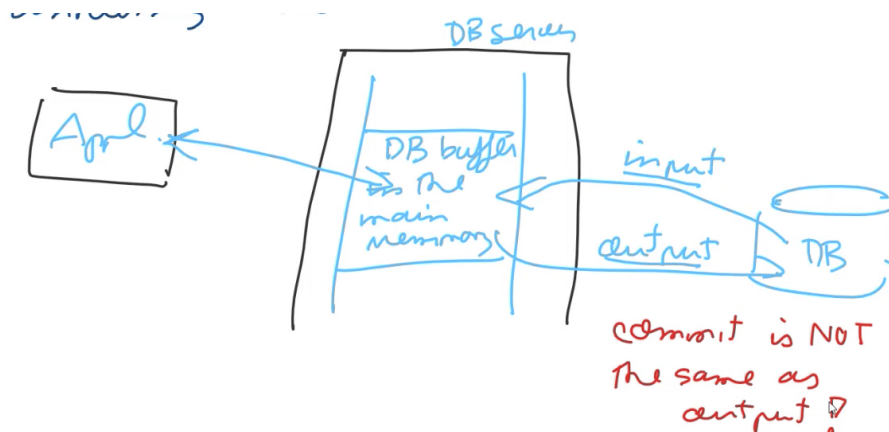
some programmer put commit on every after sql statement, why? avoid the lost of data if failure occur → damage the whole idea of transaction.



commit does not mean to move the modified data from the main memory to the mass storage.

Example 1: Deposit an amount to a university account. (**commit before output**)

- when our application work with transaction with database, it never work directly with the mass storage (disk). Application always work with the database buffer in a main memory.



- DB server have database buffer in a main memory, and you have your application, doesn't matter if it run in the same machine or not.

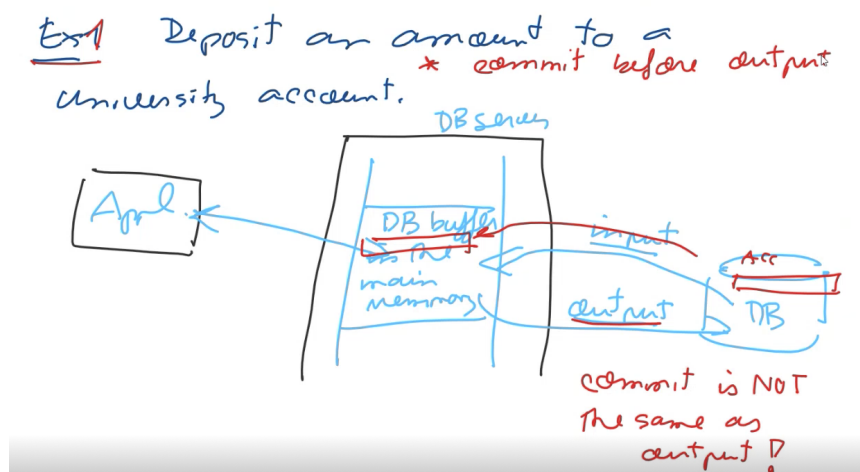
- Application always interact with database buffer, not database on the disk.
 - In the case that your application require any data, the dbms will search inside the DB buffer.
 - if found will return result to your application.
 - if not, it will load data from database space to the buffer (input activity). (load data from mass storage to DB buffer)
 - sometimes, when we no longer use data in the buffer, it can be output back to the space to clear the space and make room for other data. (output)
- from disk to main memory → input
- from main memory to disk → output

👉 commit is not the same as output.

people who code incorrectly, their intention is to output.

What if you want to pay some fee to kmitl, and you the first one that want to pay, but the kmitl account is not yet in DB buffer.

So, you want to transfer, DBMS need to input the data to DB buffer. (kmitl acc from disk to DB buffer)



- And your application deposit money to this account
- so it involved input and modified data in main memory (you the first one)

- another student come and pay → transfer to DB buffer
 - after each commit, data modified buffer may not be output to the disk.
 - WHY? because the data is still busy (people still using it), so they no point to output the content of the buffer for each and every transaction, the disk will be too busy and slow. The thing that still be in used should be in a main memory, modified them and when finished → dumped to the disk together. NOT EVERY COMMIT MEAN OUTPUT. (maybe to avoid too many disk access)
 - this is case where commit before output.

And you can imagine if the system crash and you have few commit in DB buffer → if system crash, you lost main memory. The challenge is how to be recover them, because they committed, so that they will be seen permanent on DB space on the disk → the challenge

Example 2 (interesting point: output before commit)

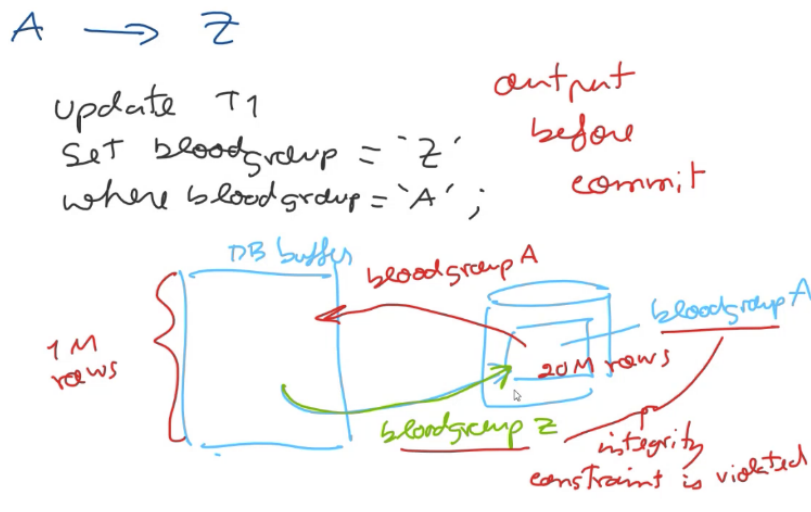
Blood group update.

Suppose we have blood group A, and for some reason WHO said this no longer A, let's called in Z. And imagine you looking through uni database, and there many student with blood group A.

And now you have to update to Z. Operation is easy.

Update T1
 Set bloodgroup = 'Z'
 where bloodgroup = 'A';

Suppose you looking through Thailand database → 20 m of people have blood group A, and you want to update it to be blood group Z.



- to change from A to Z, DBMS must read 20M row into buffer, and suppose DB buffer is smaller than a disk, and we can fit something like 1M at a time.
 - so, the first 1M to from the disk to main memory, modified A now become Z and what's next? you need to read the next batch, the Z one have to be output back to the disk.
 - On the disk, you have blood group A with Z together.

Integrity constraint → you either have blood group Z or blood group A, you should not have A, Z together. but in practice, now we have both Z and A together.

- Violation of integrity constraint

Output before commit

- You have to force data out even before commit.

Problem is what if the system went down during this operation, you have some Z value output to the disk already → you have A, Z in disk → Challenge, how to remove those uncommitted data like Z out of the DB space.

Do not mix commit(logical) with output(physical).

case 1: have many commit before output

case 2: have output, modified data to the DB space already before commit.

challenge → **system failure**

case 1: how could we recover those committed transaction, committed already
loss in memory

case 2: failed → cancelled

- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions

Who will take care of this failure → DBMS not application program (transaction recovery)

Concurrent execution of multiple transactions → you have many transaction running, how can you know the result is correct.

What is correct?

- will discussed in separate chapter
-

ACID Property (not definition of transaction)

ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.



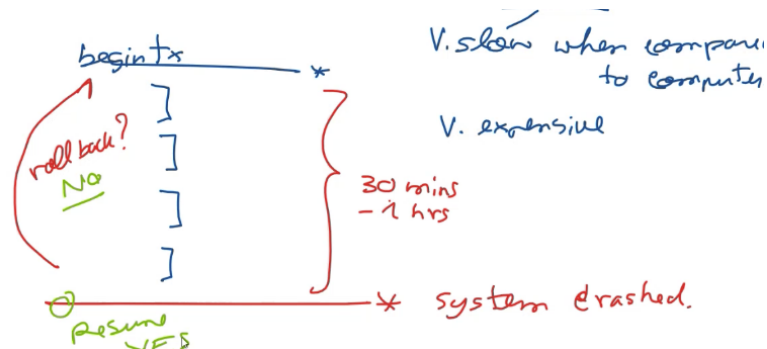
- Good property, recommended property

Some transaction may not have ACID property

- **Atomicity**

- Either all operations are succeeded or all failed
- Except for long duration transaction
 - What is a long duration transaction?
 - long duration transaction — involve human intervention (case 1, 2 → no human intervention)
 - when we have human intervention, interactive user, interactive user and human are consider slow I/O, we people are very slow compare to computer for example, the access speed could be milisecond, but for us to key something, it could take few second. A screen full could take few minute possible, this is why we call in long duration.
 - Not only very slow, but also very expensive. → cost of human is expensive.
 - So in principle with long duration transaction, the entire transaction should not fail in atomic manner. Instead with a long

transaction duration with lot lot of human involvement should failure occur, after failure, the system should zoom back to the failure point or nearly failure, not to cancelled everything.

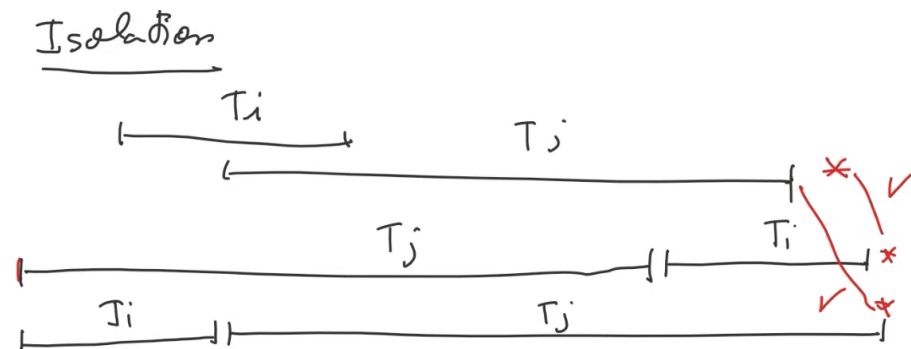


- example, suppose a group of friend would have to have a customized tour → discuss with tour agency to create a tour, tour agency begin transaction → ticket, hotel, etc. → after 30 minutes - 1 hour, system crashed. Do you want whole system to roll back? it took long time to discuss, no one want that. No, we want to keep activity and resume
 - in the transaction, we have many operation like withdraw or deposit.
 - this is consider one unit, either it all failed or it all success. We don't something that half done.
- **Consistency**
 - Execution of a transaction in isolation preserves the consistency of the database.
 - If you have transaction, we allow integrity constraint to be violated inside, and after it back database should be consistent.
 - Exception: in the case of distributed database, suppose you have more than 1 site, you may have duplicate data. Many copy of the same database in many server, many places around the world for example. It possible that we not preserve immediate consistency. Now we have something called **"Eventual consistency"**, You update one site, another copy not correct yet, but eventually all with be update and correct by the system one by one. They don't have to be all updated in real time. This help reduce the cost, update and commit one by one.
- **Isolation**

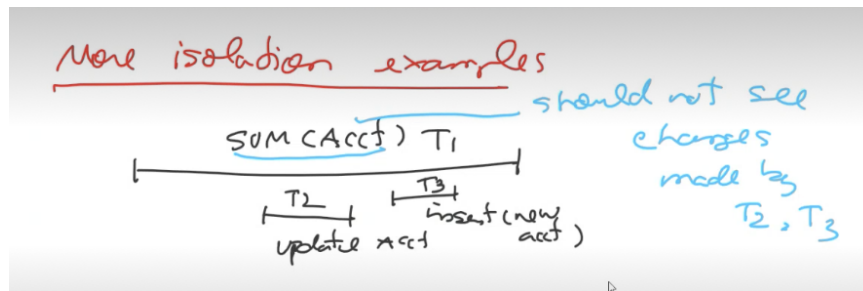
□ **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.

□ That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.

- They run separately, they don't see each other.
- They have many user running, transaction of those user don't see each other.



- Exception, what if you want to do something, and you want to see the intermediate result of another process. You don't want this isolation
- for example, project engineer analyst the data of the project, and the data entry operator key in the data all the time, the analyst would like to see the realtime data input which are not yet commit. Based on Isolation analyst, engineer won't see the newly entered data because it not yet commit.
- Nowadays, you can set isolation level to make sure you can have what you want?
 - sometime you don't want to be disturbed by new data while analyst
→ you can said i don't want to see
 - if you want, you can said i want to see newly enter data.
- More Isolation Example



- should T_i see all this changes
 - In principle, T_i should not see all this changes → T_i start first. We prefer T_i first, T_j later. (This sum should not see changes made by T₂ and T₃)
 - In practice, some analyst analyze the data and people enter new data, what if analyst want to see the changes (real time change). Based on this, you won't see them. → Exception
- Isolation Level

Weak Levels of Consistency

- Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
 - E.g. a read-only transaction that wants to get an approximate total balance of all accounts
 - E.g. database statistics computed for query optimization can be approximate (why?)
 - Such transactions need not be serializable with respect to other transactions
- Tradeoff accuracy for performance

Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read, repeated reads of same record must return same value. However, a transaction may not be serializable — it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read, but successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.
- Lower degrees of consistency useful for gathering approximate information about the database

- You can set this as a isolation level default of a DBMS, you can set it as a transaction level, connection level.
- different application may want different isolation level → this can be set.

- **Durability**

- After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.
- Design property, Without exception
- complete(commit) successfully, change permanent.
- Don't confused it with output.

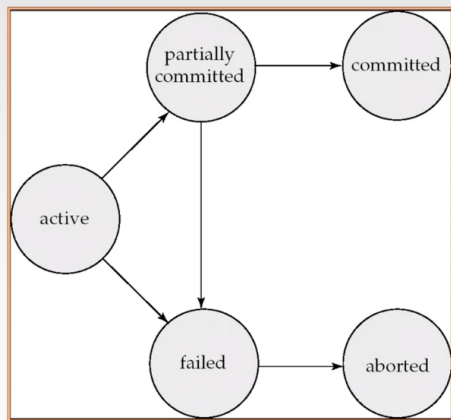
ACID property is good property of transaction, they have nothing to do with relational database model. Many NoSQL book said that relational require ACID transaction → Not true. ACID property have nothing to do with data model.

Transaction State

Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - restart the transaction; can be done only if no internal logical error
 - kill the transaction
- **Committed** – after successful completion.

Transaction State (Cont.)



- Active - initial state, the transaction is now being executed.